

Neural Language Style Transfer With StyleTransfer

📅 Updated January 10, 2024 • ⌚ 16 min read



DataChef

DataChef, founded in 2019, is a boutique consultancy specializing in AI and data management. With a diverse team of 20, we focus on developing Data Mesh, Data Lake, and predictive analytics solutions via AWS, improving operational efficiency, and creating industry-specific machine learning applications.

TAGS

#style-transfer

#neural-networks

#large-language-models

☰ Contents

StyleTransfer is a natural language processing solution developed in-kitchen by the machine learning team at DataChef.

Stylometry

The Da Vinci Code, Angels & Demons, The Lost Symbol, Inferno. They all start with Robert Langdon finding himself in a mystery he hadn't signed up for. They all involve historical monuments, deaths and treacheries. However, they also have something else in common: they are best-sellers.

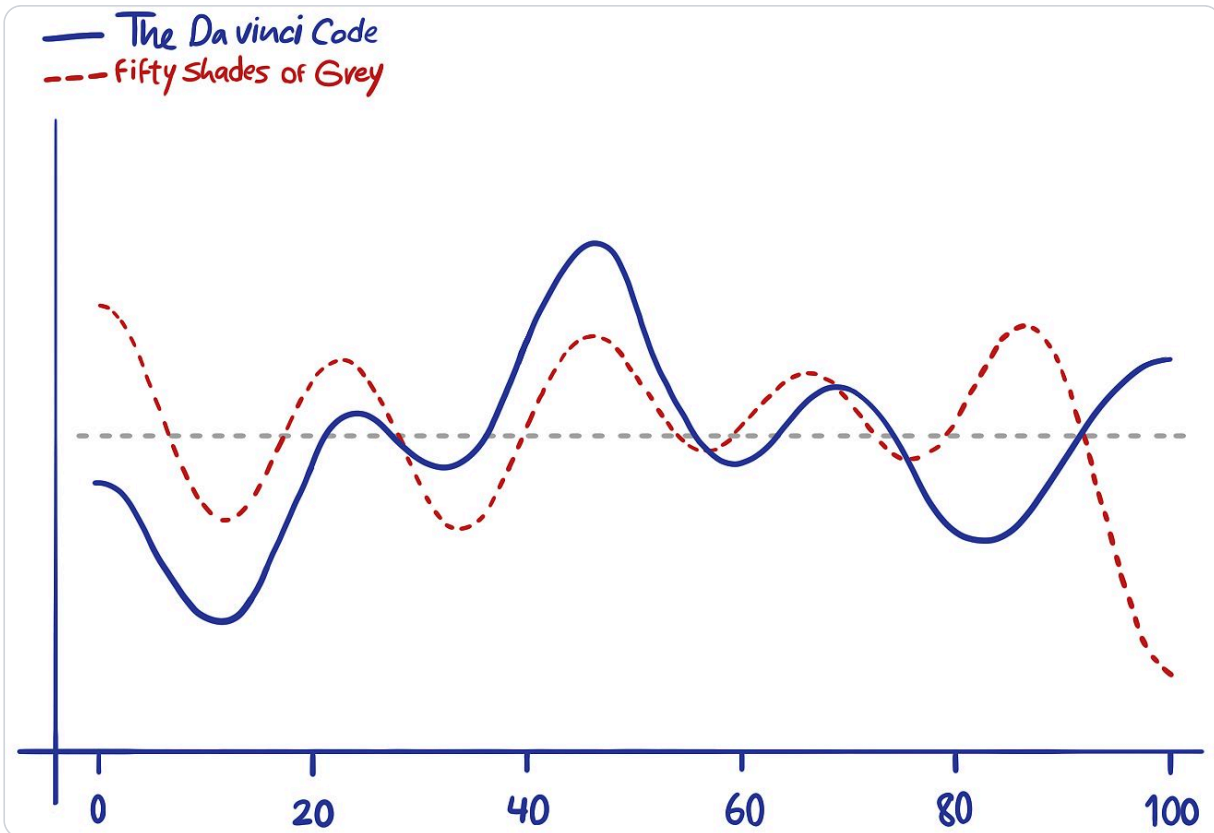
When renowned American novelist Dan Brown announces a new book, readers blindly place preorders even if they are aware that it is likely just another work of fiction wrapped around other works. So what is this mysterious ingredient in his mysteries that hooks the readers? More generally, how can one go about teasing out the style characteristics of any writer in a quantified manner?

Motivated by these questions and inspired by the potentials of the modern language models, our team of scientists and engineers at DataChef undertook project StyleTransfer. We soon realized that the above question was too wild to lend itself to universal black-box methods.

Check out [Data vs. Dan Brown](#) episode of FTFY Podcast

'Meow' means 'woof' in Cat. George Carlin

At our first stop, we dove into *The best seller code*, a book by Jodie Archer and Matthew Jockers. Their book is an account of applying the classic machine learning techniques to decide if a new manuscript is likely to land on the best seller lists. They find that hitting the emotional ups and downs at the right frequencies across the entire plot line is far more important than producing high quality prose. For instance, comparison of the plot line between 50 Shades of Grey and Da Vinci Code reveals an astonishing level of similarity in this regard.

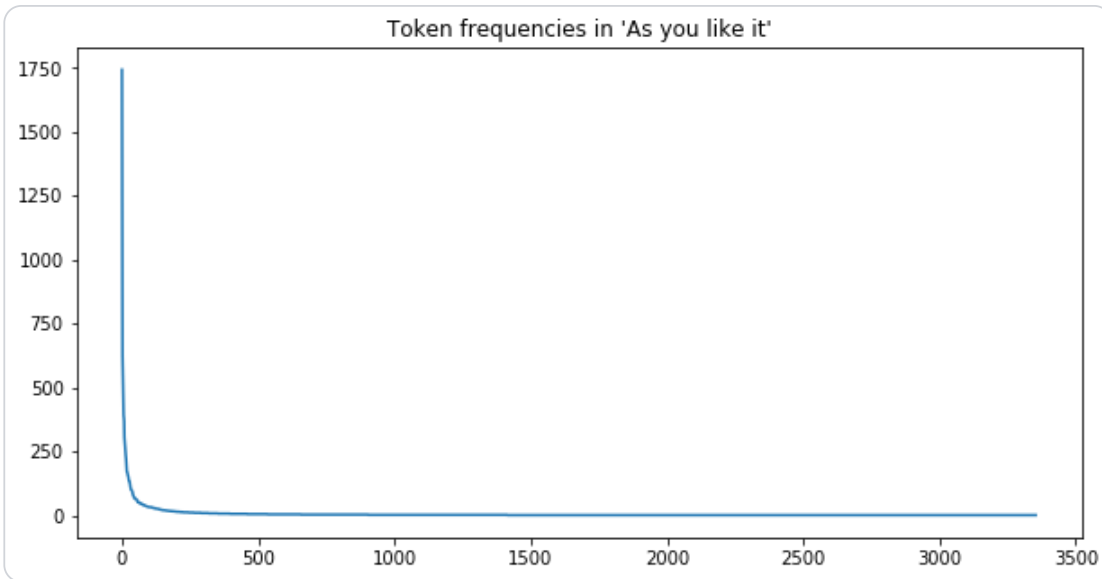


A comparative visualization of the emotional ups and down in the storylines.

On a more micro level, it's also quite astounding to note that a good amount of the author's finger print is captured by their use of words such as: a/an, the, of, I, s/he, etc. As noted in the book, J. K. Rowling who published a crime fiction in 2013 under the pseudonym of Robert Galbraith, would come to realize that it's very difficult to alter or disguise one's linguistic fingerprint.

Period points are also more common in winning prose, and both semicolons and colons are significantly less so. The best seller code

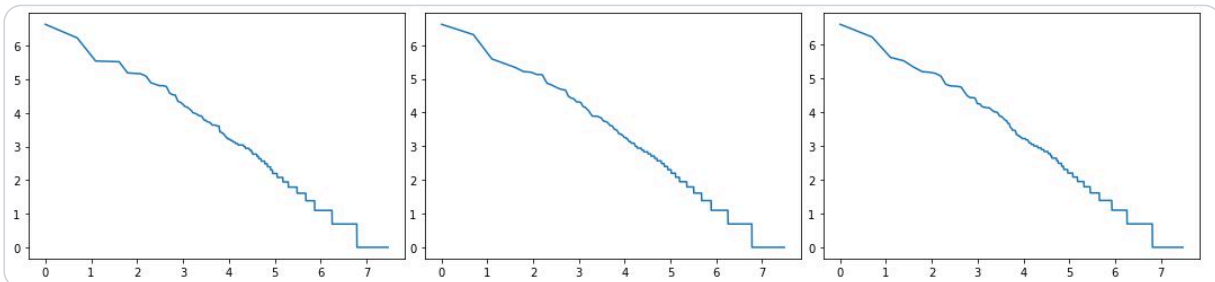
A closer look into the literature revealed that combinatorial statistics have a long standing connection to stylometry which is a branch of linguistics with focus on the quantified aspects of language style. In particular, we found the works of George Zipf in the 40s remarkably insightful. He had noticed that the frequency of a word in a corpus is closely tied with the inverse of its rank. Roughly speaking, this is to say that the frequency of the second (third) most common word in a corpus is half (a third) the frequency of the first word, etc.



Zipf plot for Shakespeare's As you like it. The x-axis corresponds to the tokens and the y-axis to counts.

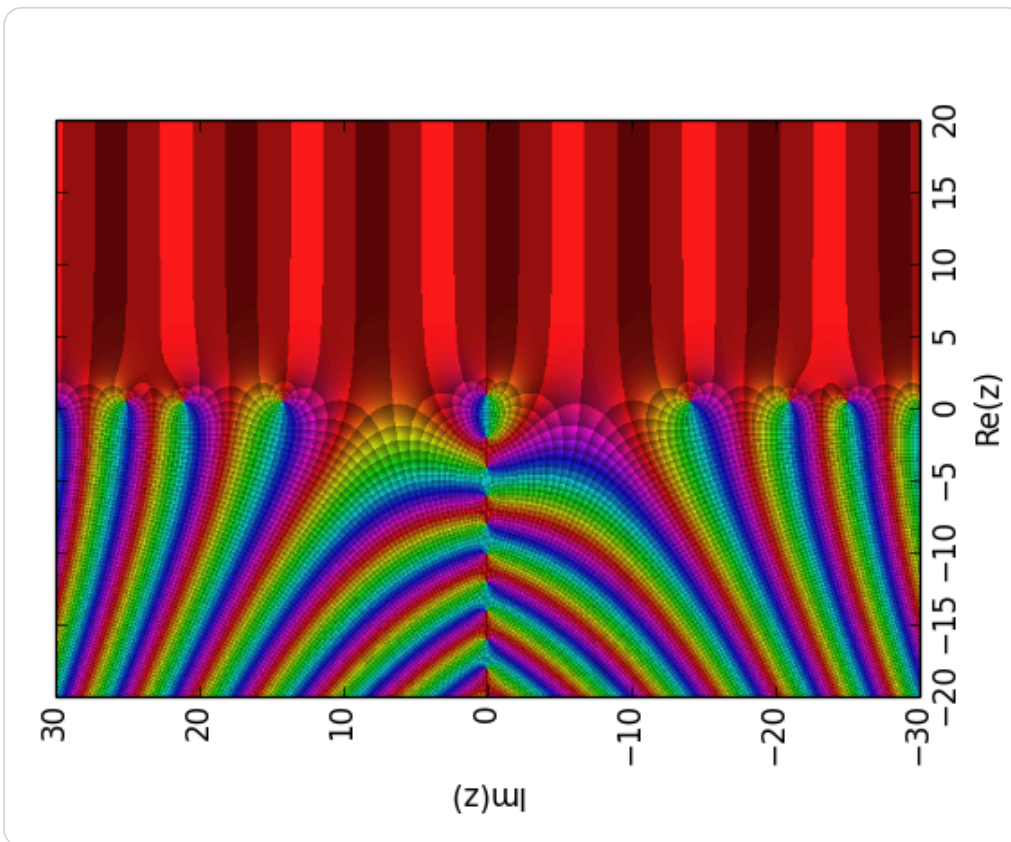
Moreover, he generalized this type of observations to other contexts such as economics, demographics, etc. This becomes less surprising upon noticing that the same ideas have been floating around under other names such as the Pareto Principle, the 80-20 rule, etc.

The following plots show the log-log version of the above plot for three random subsets of the text from *As You Like It* by Shakespeare. Notice that the plots tend to become straight lines of very similar slope around the center.



Empirical evidences strengthened the promise that these fractal like properties did in fact have something to do with the finger print of the author. There is a profusion of evidence that this kind of analysis could be utilized to detect even more global qualities such as the language of the text, dialect, genre, etc.

In particular, these ideas were further developed by Benoit Mandelbrot from a mathematical stand point and served as motivation for the theory of fractals. Zipf-like distributions arise quite naturally in the modern data science as datasets that are innately unbalanced. Mathematically, these distributions motivate a closer look into Dirichlet series and their analytic continuation: the Riemann Zeta function.



The Riemann zeta function represented through domain colouring. Image credit: Empetrisor.

In the technical jargon of computational linguistics, the type of questions that we are considering belong to stylometrics. One notable application of stylometrics is author attribution. For instance, it is revealed that a sizeable chunk of Shakespeare's works were delegated and written by other collaborators due to various reasons. Of course, the scope of the applications is endless and we believe that we are at the sweet spot where we can merge the classical techniques with the modern methods in natural language processing that have emerged in the last few year.

StyleTransfer

StyleTransfer is an ambitious project that aims to facilitate style transfer for text. This is a challenging task that by its nature lacks a clear recipe within the framework of modern natural language processing. As we shall see below, our first objective in the first phase is to tune our models to measure style similarity between a given input text and a corpus (as the source of style).

What is style?

Roughly speaking, style is any linguistic characteristic that is independent from the content of writing or speech. This includes the lexicon of the author, the grammatical structure in use, punctuation, the pace and order at which topics are introduced and expanded, the types of

narration and argumentation, the element of storytelling and so much more. In this sense, style is a delicate linguistic characteristic that varies even during the writing career of an author.

Style transfer?

Consider the problem of applying the style of a known corpus to another piece of writing. To be more specific, imagine that you're writing an application letter to your desired schools, funding agencies, prospective employers, etc. Let's say that each recipient requires or expects a certain style of writing that emphasizes certain aspects of the letter. Note that this requirement can easily include the language in which the letter gets written too.

Style also brings major implications for copy writing and marketing. Imagine that you could transform your initial copy to a version that is expected to have better resonance for any other target demographics or audience. How would that transform social media marketing?



Image style transfer: from a rooster to pico de gallo (rooster's beak).

Anyways, let's get back to reality for now. Style transfer for images is fairly well understood [Gatys et al., 2016], however language is a lot more delicate and complex. As a result, a direct transition from image processing techniques to language processing solutions isn't quite feasible. Most modern attempts at tackling language style depend on heavily supervised models that arrive at style by leveraging vast amounts of data. The dominant approach among these relies on the sequence to sequence translation techniques which mostly target rudimentary aspects of the style such as positive/negative, news/academic, etc.

How to measure style similarity?

Let's consider a simpler question. Let's say you're given a large enough corpus of fixed style. Now, for any input text, assign a similarity score relative to the style source. To this end, we are looking for simple and effective methods of quantifying the similarity of prose or poetry in the local sense that respects qualities such as entropy, length of tokens and sentences, punctuation and so on but is likely to be blind to the overall ups and downs of the text or the order in which the topics appear and is ideally invariant of the genre.

Before diving into the details, let us take a step back to point out that we can consider language as a channel of processing and transferring information (or more specifically,

code). From the perspective of information theory, a shorter code is superior as it allows the channel to carry more information. However, channels are subject to noise. In the daily human context, noise includes any perceived vagueness, misspeak, misdirection, interference with other channels of communication and so on. As in general with any code transference, redundancy can be introduced or demanded to facilitate the decoding process. For humans, this process is inherently asymmetric as inference tends to be cheap but articulation expensive.

As our first attempt, we followed the footsteps of George Zipf. One of his insights was that more frequent words tend to be shorter to allow for a more efficient packing of information. More generally, it is believed that words that are more predictable given the preceding context, also tend to be shorter to allow for a more efficient packing of information.

Our initial idea was to simply mask the entire text and only keep the length of tokens. This in effect turns the entire text into a time series of integers. Now, if we believe Zipf's idea that combinatorial statistics is all we need to determine style, we can linearize his distributions and arrive at a sequence model. So, we trained an LSTM model to take a few token lengths and produce the next one.

The long-short term memory is a deep neural network that attempts to create a memory retention effect while keeping the vanishing gradient phenomenon under control.

We wrote the first iteration of this model in PyTorch and trained it on *Alice's Adventures in the Wonderland*. As a measure of accuracy, we considered the percentage of token lengths that were predicted correctly. Given the fuzzy nature of language, it is difficult to formulate precise requirements for evaluating such a model. We decided to let the model run until we got about 80% train accuracy. Then, for evaluation, we gave it random pieces of different books and noticed that the average score was around 55%. It is worth noting that for a few of these random texts, we got scores that were higher than training accuracy average. This could be partly due to the fact that our style source is a short book.

Alice : I simply must get through!
Doorknob : Sorry, you're much too big. Simply impassible.
Alice : You mean impossible?
Doorknob : No, impassible. Nothing's impossible.

We grabbed our corpora from Project Gutenberg in the text format and performed some manual cleaning and preprocessing such as removing the header and footer, chapter numbers, special characters and so on. Then, we used 0-1 encoding for each token length up to maximum length.

We chose to use batches of length 5 and let the model predict the length of the next token and put cross entropy as the loss function. Given the nature of the problem, we only applied

very light regularization as we wanted to the learned model to be slightly overfit. Finally, we normalized the the accuracy of the test text by the expected accuracy on the train text to arrive at the desired similarity score.

There are many possible improvements in this setting. For instance, one could enrich the input sequence by attaching the part of speech tag of each token to its length. Alternatively, one could employ a convolutional network at tackle this question to get similar results.

Transformers

Next, we moved to transformers to obtain generative solutions. Transformers are novel deep neural networks that in a sense evolved out of recurrent neural networks and hidden Markov models. Cognitively, the transition from RNNs to transformers corresponds to the transition from memorization to attentiveness. As a result, transformers are more flexible in the sense that the data no longer needs to be fed sequentially.

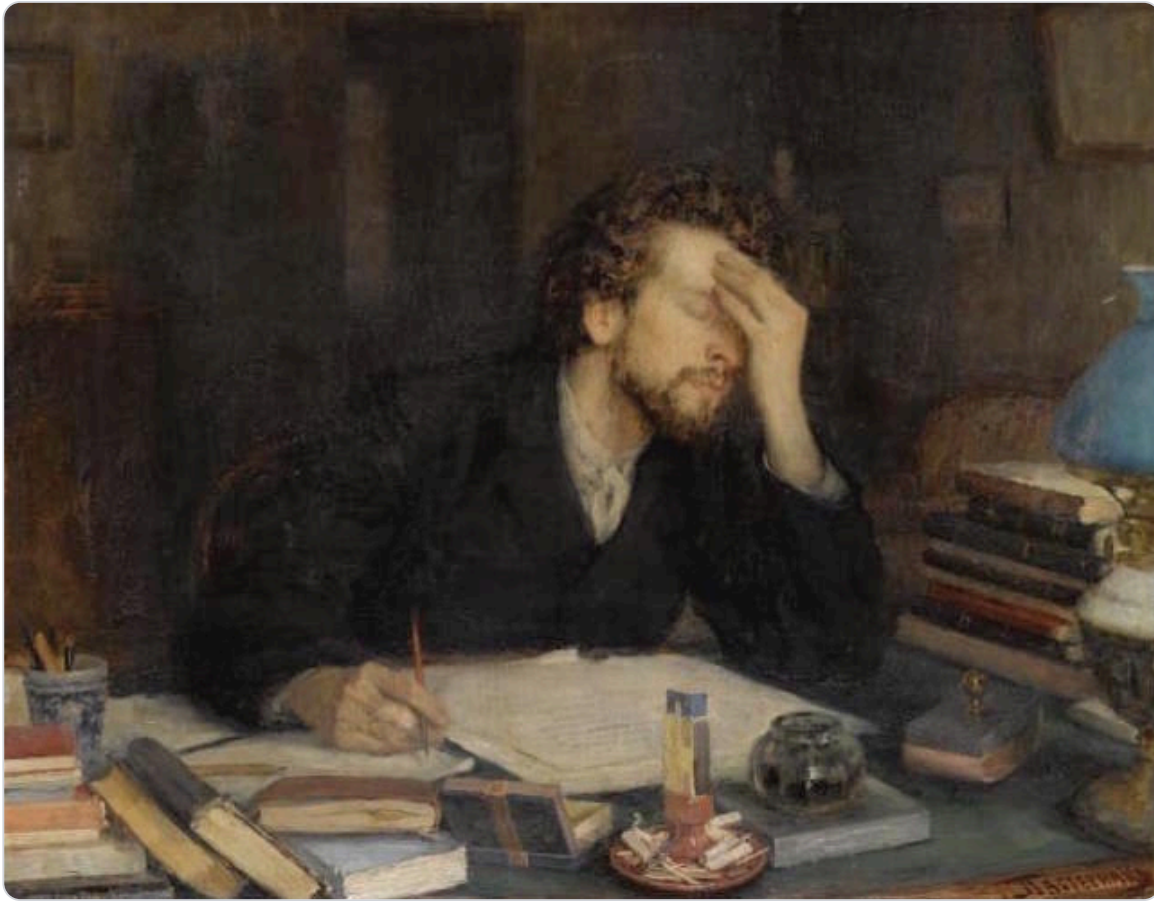
The crucial component of transformers is their attention units. As with almost any language model, the transformer architecture is designed to create a (vector) representation of the words in the language. The attention units are there to seek out relationships between these word-representing vectors at various levels of granularity. This induces a topology on the cloud of points of a language model that mimics the language structure.

During the not terribly long time since their introduction in 2017, transformers have proven to be successful in various domains such of text generation, text summarization, translation, etc. Moreover, many of these techniques are trickling down into other areas of machine learning which is a fascinating phenomenon.

In the rest of this post, we are going to give an account of two of our transformer based generative models. We trained our models on a ml.g4dn.xlarge accelerated computing instance on AWS SageMaker which carries a 4 core CPU, a GPU and 16 GiB of memory. However, once training was complete, we were able to evaluate the models on a regular laptop.

Text completion with GPT-2

As a creative writer, you must be all too familiar with the writer's block of various levels of intensity. For example, it could be that you know where you want to go with your writing but you're not quite pleased with the words coming out of your mind. Well, it'd be only natural to employ the machine to give you boosts here and there. It's almost as if you're favorite writer is sitting by your side to mentor you through the process.



A depiction of the writer's block by Leonid Pasternak

You can live with me in this house I've built out of writers blocks. Peter Wentz

For our purpose, we used the GPT-2 transformer published by OpenAI. To create our first proof of concept, we decided to go with the transformer library by Hugging Face 😊 which provides a convenient API for working with many of our favorite transformer models.

As the source of style, we picked:

- *Alice's Adventures in Wonderland* by Lewis Carrol
- The entire content of the *Coding Horror* blog series by Jeff Atwood

Given that we used GPT-2 under Hugging Face, we won't spend much time here to linger on the technical details. Instead, we are first going to showcase some completions following the seed sentence: '*Do Italians put pineapples on pizza?*'

- Style source: Alice's

"Do Italians put pineapples on pizza?," said the Lizard. "I don't know," said the March Hare. "Then you shouldn't have put them on," added the King. "They're too stiff". "They're too supple!" the March Hare went on in a deep voice, "They're too full of themselves to be anything but a pack of cards: let them be gone before the trial's over ... "

- Style source: Coding Horror

Do Italians put pineapples on pizza? I'm not sure. I mean, it's not that hard, necessarily, but I think one particular American style of pizza does it better. The pineapples are one of those things that puts all its complexity in perspective: more like, say, milk chocolate than a computer. One of the peculiarities of the pea is that it's very adaptable: it can synthesize (almost) anything in a single pass.

Evidently, the machine's imagination tends to run wild the more it gets away from the seed. This could be partly remedied by the using the descendent of GPT-2, namely GPT-3, which enjoys a much larger parameter space. As a second experiment, we put the following seed: 'I wonder if arts are artificial'.

- Style source: Alice's

"I wonder if arts are artificial like machines?" said Alice, who always took a great interest in questions of fashion. "I haven't the slightest idea," said the Footman, "but they're looking at me like telescopes."

- Style source: Coding Horror

I wonder if arts are artificial in the way that computers are constructed. Perhaps in a very real way. Maybe an entire generation of programmers will be trapped in a world of computers without an artistic expression, without the ability to express creative ideas online.

The following screencast shows our interactive web app wrapper for our GPT-2 model written in Flask. This instance is only hooked to the plain English model for now.

A demo of the GPT-2 sentence completion module along with the LenLSTM style similarity measure relative to Alice's Adventures.

Text rephrasing with T5

In this final section, we'll go over a rephraser module that was built on top of the Text-to-Text Transfer Transformer model. For training data, again we went back to our favorite English dramatist, William Shakespeare, and scraped the web for a dataset of 23 plays with one-to-one translations into modern English. What follows is our machine's attempt at Shakespearifying the lyrics of *My heart will go on*.

Original: Every night in my dreams

Rephrase: I dream all night in my dreams

Original: I see you, I feel you

Rephrase: I hear thee, I feel thee

Original: That is how I know you go on

Rephrase: So I know thou movest on

Original: Far across the distance

Rephrase: Far out of the farthest, far out of the distant distance

Original: And spaces between us

Rephrase: Between us there is a gap of space

Original: You have come to show you go on

Rephrase: Thou art come to make sure thou shalt go on

Original: Near, far, wherever you are

Rephrase: Near, far, where thou art

Original: I believe that the heart does go on

Rephrase: I believe that the heart goes forth

Original: Once more, you open the door

Rephrase: One more time, thou openst the door

Original: And you're here in my heart

Rephrase: And thou art herein my heart

Original: And my heart will go on and on

Rephrase: And heart, and heart, and heart, and heart, and heart

Original: Love can touch us one time

Rephrase: Love may once touch us

Original: And last for a lifetime

Rephrase: And last for ever

Original: And never let go 'til we're gone

Rephrase: And let never be gone 'till our time, 'till there is a time in the sky

Original: Love was when I loved you

Rephrase: Love was when I loved thee

Original: One true time I'd hold to

Rephrase: One I would hold to, though, by and by

Original: In my life, we'll always go on

Rephrase: In my life we shall be steadfast

Original: Near, far, wherever you are

Rephrase: Nah, far, whoever'st thou

...

As the name suggests, T5 has the convenient feature that the user interface is solely text based. We collected our data as json objects and thereafter compiled everything into a csv file with two columns: original and modern.

```
{"original": "Give me your hand. Art thou learned?",  
  "modern": "Give me your hand. Are you educated?."}
```

For the above demo, we fed the lines one by one to our fine-tuned T5 model and asked for up to 10 outputs. The settings were as follows.

```
outputs = model.generate(  
    input_ids=input_ids, attention_mask=attention_masks,  
    do_sample=True,  
    max_length=256,  
    top_k=100,  
    top_p=0.95,  
    early_stopping=True,  
    num_return_sequences=10  
)
```

Text rephrasing with GPT-2

GPT-2 is typically recognized for its generative capabilities. However, we wanted utilize its sequence-to-sequence nature and employ it for the rephrasing task. To this end, we reformatted our Shakespeare dataset in the following format where we used s and p tags to distinguish between the two sides.

```
<s>Why, the one I just told you about.</s> >>>> <p>Why, this that I speak o  
<s>Me too, I swear.</s> >>>> <p>Or I, I promise thee.</p>
```



After training we fed the model the sentence we wanted paraphrased in the s-tags. The output contained the »» and another sentence contained in p-tags. By cleaning the output we had a paraphrased sentence.

We are happy to report that this approach worked successfully and produced results that were of comparable quality to the ones shown above. Though T5 did a better job.

Let's recap. What is StyleTransfer useful for?

Well, there are numerous products that can be built on top of StyleTransfer or as spinoffs. Here, we barely scratched the surface by implementing a few generic components. Let us close this post by recalling the items we already discussed as possible use cases:

- Rewriting job descriptions, resumes, letters, etc.
- Tuning and harmonizing the tone and voice in copy writing.
- Rewriting and improving captions for social media contents.

What would you build on top of StyleTransfer?



Product Managers and the Art of Engineering a 'Yes'

If you spend any time reading about product management online, you'll frequently encounter a piece of advice: saying no is the product manager's superpower. There are...

Jan 19, 2026 ⌚ 6 min read



Subscribe to the newsletter.

Get new posts in your inbox.

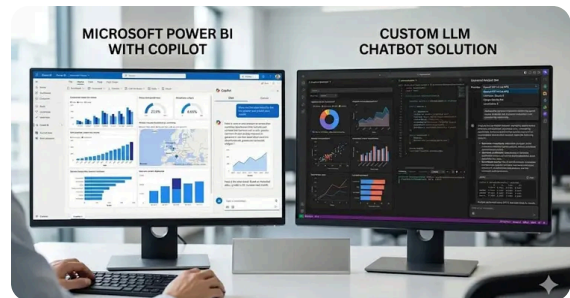
you@example.com

Subscribe

Copilot VS. Custom LLM: Navigating the Generative BI Landscape

For years, Business Intelligence has followed a familiar, often frustrating pattern. An executive has a question, an analyst hunts for the answer, engineers build data...

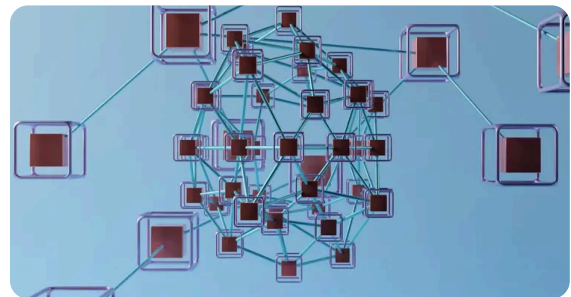
Dec 16, 2025 ⌚ 8 min read



Is Your Organization Ready for Data Mesh? A Practical Readiness Check

You are being asked to make a long-term bet on data architecture. Most conversations frame this as a choice between data lake and data mesh. Vendors, internal team...

Dec 11, 2025 ⌚ 11 min read



Beyond Dashboards: An Executive Introduction to Generative BI

For years, leaders have relied on dashboards and BI teams to understand what is happening in their organization. The rhythm has been familiar: you notice something unusual,...

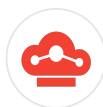
Dec 4, 2025 ⌚ 6 min read



How We Turned Onboarding Into Something Magical Using n8n

Bringing a new colleague into DataChef has always felt a bit like welcoming someone into our kitchen. You want the place to feel warm and ready, and you want their first...

Dec 1, 2025 ⌚ 4 min read



DataChef

84 posts published



© 2026 DataChef

[Members](#) [Archive](#) [Privacy](#) [Terms](#)

 [Sitemap](#)  [RSS](#)

 **Hashnode**